

پروژه شماره 3: جستجوی خصمانه در بازی اتلو

پیاده‌سازی عامل‌های Minimax و Alpha-Beta Pruning

مریم موسوی | ستایش اسدیان فیلی

درس هوش مصنوعی | دانشکده ریاضی و آمار | دانشگاه اصفهان | نیم‌سال دوم 1404-1405

استاد: دکتر فریا نصیری مفخم

لینک رپازیتوری گیت (پابلیک): <https://github.com/SmithyOrFalse/0thello>

1. توضیح روش پیاده‌سازی

در این پروژه سه بخش خواسته‌شده پیاده‌سازی شد: تابع ارزیابی (evaluate)، عامل MinimaxAgent و عامل AlphaBetaAgent. ساختار فایل‌های محیط بازی (پوشه‌ی game/) و عامل‌های پایه (RandomAgent و GreedyAgent) بدون تغییر باقی ماند و فقط دو فایل agents/minimax_agent.py و agents/alphabeta_agent.py تکمیل شدند.

نمایش بازیکنان مطابق محیط است: BLACK = 1 و WHITE = -1، بنابراین حریف هر بازیکن با opponent = - می‌باشد. این قرارداد امکان می‌دهد الگوریتم به صورت یکنواخت و بدون شرط‌های جداگانه برای سیاه و سفید نوشته شود.

ساختار کلی الگوریتم

هسته‌ی هر دو عامل یک تابع بازگشتی است که درخت بازی را تا عمق مشخص depth پیمایش می‌کند. برای تولید فرزندان، از تابع copy() محیط استفاده می‌شود تا نسخه‌ی مستقلی از وضعیت ساخته شود و حرکت روی آن اعمال گردد؛ به این ترتیب وضعیت اصلی بازی هرگز تغییر نمی‌کند. شرط‌های توقف بازگشت عبارت‌اند از: رسیدن به عمق صفر یا پایان بازی (game_over()). در این نقاط، مقدار وضعیت با تابع ارزیابی از دیدِ root_player (بازیکنی که در ریشه‌ی درخت تصمیم می‌گیرد) محاسبه می‌شود.

مدیریت «پاس» (Pass)

در اتلو ممکن است بازیکنی هیچ حرکت قانونی نداشته باشد. در این حالت نوبت بدون اعمال حرکت به حریف می‌رسد. در پیاده‌سازی، هرگاه get_valid_moves لیست خالی برگرداند، بدون تغییر صفحه، فراخوانی بازگشتی برای حریف انجام می‌شود (با کاهش یک واحد از عمق) و None به عنوان حرکت برگردانده می‌شود. این رفتار با حلقه‌ی اصلی موجود در tournament.py که در هر نوبت بازیکن را عوض می‌کند سازگار است.

سازگاری با اندازه‌های مختلف صفحه

طبق هشدار صورت پروژه، محیط ارزیابی لزوماً 6×6 نیست. به همین دلیل تابع ارزیابی به گونه‌ای نوشته شد که برای هر اندازه‌ی صفحه کار کند: ماتریس وزن موقعیتی برای 6×6 به صورت دستی تنظیم شده و برای سایر اندازه‌ها به صورت پویا تولید می‌شود (گوشه‌ها بیشترین وزن، خانه‌های مجاور گوشه وزن منفی، لبه‌ها وزن مثبت کم، و خانه‌های داخلی کمترین وزن). همه‌ی معیارها نیز نرمال‌سازی شده‌اند تا مستقل از تعداد خانه‌ها معنادار بمانند.

2. توضیح تابع ارزیابی

تابع `evaluate(game, player)` کیفیت یک وضعیت را از دید بازیکن موردنظر تخمین می‌زند؛ مقدار بزرگ‌تر یعنی وضعیت بهتر. این تابع ترکیبی وزن‌دار از چهار معیار شناخته‌شده‌ی اتلو است که هرکدام در بازه‌ی تقریبی $[-100, 100]$ نرمال شده‌اند:

- اختلاف مهره‌ها (Coin Parity):** اختلاف نسبی تعداد مهره‌های بازیکن و حریف. در اواخر بازی تعیین‌کننده است اما در ابتدای بازی معیار گمراه‌کننده‌ای است.
- امتیاز موقعیتی (Positional):** جمع وزن خانه‌های اشغال‌شده بر اساس ماتریس وزن. گوشه‌ها (پایدار و غیرقابل‌برگشت) وزن بسیار بالا و خانه‌های مجاور گوشه وزن منفی دارند.
- تحرك (Mobility):** اختلاف نسبی تعداد حرکات قانونی دو بازیکن. محدودکردن گزینه‌های حریف یک مزیت راهبردی قوی است.
- تسلط بر گوشه‌ها (Corner Ownership):** اختلاف تعداد گوشه‌های تصاحب‌شده. چون گوشه‌ها هرگز برگردانده نمی‌شوند، اهمیت ویژه‌ای دارند.

$$\text{score} = w1 \cdot \text{parity} + w2 \cdot \text{positional} + w3 \cdot \text{mobility} + w4 \cdot \text{corner}$$

نکته‌ی کلیدی، وابستگی وزن‌ها به مرحله‌ی بازی است. با محاسبه‌ی نسبت خانه‌های پرشده (`progress`)، در اوایل/اواسط بازی به تحرك و کنترل گوشه‌ها وزن بیشتری داده می‌شود و اختلاف تعداد مهره‌ها کم‌اهمیت تلقی می‌شود؛ اما در اواخر بازی وزن اختلاف مهره‌ها به شدت افزایش می‌یابد چون نتیجه‌ی نهایی بر همین اساس تعیین می‌شود:

مرحله بازی	اختلاف مهره	موقعیتی	تحرك	گوشه
اوایل/اواسط ($\text{progress} < 0.55$)	5	10	15	35
اواخر بازی ($\text{progress} \geq 0.55$)	25	8	8	35

همچنین اگر وضعیت، پایان بازی باشد، تابع به جای تخمین، نتیجه‌ی قطعی را با مقداری بزرگ (± 10000) به علاوه‌ی اختلاف مهره‌ها) برمی‌گرداند تا الگوریتم برد قطعی را بر هر تخمینی ترجیح دهد.

3. نتایج آزمایش‌ها

هر عامل در برابر RandomAgent و GreedyAgent در 20 بازی آزموده شد. برای انصاف، عامل ما به صورت متناوب نقش سیاه (شروع‌کننده) و سفید را بازی کرد. نتایج:

عامل	حریف	تعداد بازی	تعداد برد	نرخ برد
Minimax (d=3)	Random	20	20	100%
Minimax (d=3)	Greedy	20	20	100%
Minimax (d=4)	Random	20	20	100%
Minimax (d=4)	Greedy	20	20	100%
AlphaBeta (d=3)	Random	20	19	95%
AlphaBeta (d=3)	Greedy	20	20	100%
AlphaBeta (d=4)	Random	20	20	100%
AlphaBeta (d=4)	Greedy	20	20	100%

عامل‌های ما در عمل تقریباً تمامی بازی‌ها را در برابر هر دو حریف پایه می‌برند. تنها یک باخت (AlphaBeta d=3) در برابر Random رخ داد که ناشی از تصادفی بودن حریف و عمق پایین جست‌وجو در آن اجرای خاص است؛ با افزایش عمق به 4 این مورد نیز برطرف شد.

4. جدول مقایسه Minimax و Alpha-Beta

برای سنجش اثر هرس آلفا-بتا، یک بازی کامل (در برابر حریف یکسان با همان دانه‌ی تصادفی) با هر دو الگوریتم اجرا شد و مجموع گره‌های بازدیدشده و زمان اجرا اندازه‌گیری گردید. هر دو الگوریتم در تمامی وضعیت‌ها دقیقاً مقدار و حرکت یکسانی تولید کردند (اختلاف مقدار = 0)، که نشان می‌دهد آلفا-بتا فقط سرعت را افزایش می‌دهد و نتیجه را تغییر نمی‌دهد.

عمق	گره‌های Minimax	زمان Minimax	گره‌های Alpha-Beta	زمان Alpha-Beta	کاهش گره
2	865	0.11 ثانیه	658	0.08 ثانیه	23.9%
3	5,971	0.75 ثانیه	3,063	0.37 ثانیه	48.7%
4	35,686	4.41 ثانیه	7,650	0.86 ثانیه	78.6%

80.6%	3.55 ثانیه	32,738	20.21 ثانیه	169,106	5
-------	------------	--------	-------------	---------	---

* اعداد مربوط به مجموع گره‌های بازدیدشده در طول یک بازی کامل است؛ زمان‌ها به سخت‌افزار اجرا وابسته‌اند اما نسبت آن‌ها معنادار است.

5. تحلیل نتایج (پاسخ به سؤالات)

1) افزایش عمق جست‌وجو چه تأثیری بر کیفیت بازی داشت؟

افزایش عمق، کیفیت تصمیم‌گیری را بهبود می‌دهد چون عامل پیامدهای دورتر هر حرکت را می‌بیند و کمتر در دام حرکت‌های وسوسه‌انگیز کوتاه‌مدت (مانند گرفتن مهره‌های زیاد در یک نوبت) می‌افتد. در آزمایش‌ها، عمق 4 نسبت به عمق 3 پایدارتر بود و تک‌بخت مشاهده‌شده در عمق 3 را حذف کرد. در مقابل، هزینه‌ی محاسباتی تقریباً به‌صورت نمایی رشد می‌کند (از 865 گره در عمق 2 تا حدود 169,000 گره در عمق 5 برای Minimax)، بنابراین یک مصالحه میان کیفیت و زمان وجود دارد.

2) Alpha-Beta چه تأثیری بر زمان اجرا داشت؟

هرس آلفا-بتا بدون تغییر نتیجه، تعداد گره‌های بررسی‌شده و در نتیجه زمان اجرا را به‌شدت کاهش داد. این کاهش با افزایش عمق چشمگیرتر می‌شود: از حدود 24% در عمق 2 به بیش از 80% در عمق 5 رسید. به بیان دیگر، آلفا-بتا اجازه می‌دهد با همان بودجه‌ی زمانی، عامل عمیق‌تر جست‌وجو کند و در نتیجه قوی‌تر بازی کند. در حالت ایده‌آل (با مرتب‌سازی خوب حرکت‌ها) آلفا-بتا می‌تواند پیچیدگی را از $O(b^d)$ به حدود $O(b^{d/2})$ کاهش دهد.

3) چه معیارهایی را در تابع ارزیابی استفاده کردید؟

چهار معیار: اختلاف تعداد مهره‌ها، امتیاز موقعیتی مبتنی بر ماتریس وزن، تحرک (اختلاف تعداد حرکات قانونی) و تسلط بر گوشه‌ها. این معیارها با وزن‌های وابسته به مرحله‌ی بازی ترکیب شدند تا رفتار عامل در ابتدا، میانه و انتهای بازی مناسب باشد.

4) مهم‌ترین عامل موفقیت عامل شما چه بود؟

دو عامل از همه مهم‌تر بودند: نخست، کنترل گوشه‌ها و تحرک در تابع ارزیابی، که عامل را از طمع کوتاه‌مدت دور و به سمت موقعیت‌های پایدار هدایت کرد؛ دوم، تنظیم وزن‌ها بر اساس مرحله‌ی بازی، که از خطای کلاسیک «جمع‌کردن مهره در ابتدای بازی» جلوگیری کرد. ترکیب این‌ها با جست‌وجوی عمیق‌تر (که آلفا-بتا آن را ممکن کرد) به برتری قاطع در برابر حریف‌های پایه انجامید.

5) نقاط ضعف تابع ارزیابی شما چیست؟

• وزن‌ها به‌صورت دستی تنظیم شده‌اند و بهینه نیستند؛ با یادگیری یا تنظیم خودکار می‌توان عملکرد را بهبود داد.

- مفهوم «پایداری مهره‌ها» (Stability) به‌طور کامل مدل نشده و فقط گوشه‌ها به‌عنوان نمونه‌ی پایدار در نظر گرفته شده‌اند؛ مهره‌های پایدارِ لبه‌ای نادیده گرفته می‌شوند.
- ماتریس وزن تولیدشده برای صفحه‌های بزرگ‌تر از 6×6 عمومی است و به‌اندازه‌ی نسخه‌ی دستی 6×6 دقیق نیست.
- آستانه‌ی مرحله‌ی بازی (0.55) ثابت است و برای همه‌ی اندازه‌های صفحه بهینه نیست.
- تابع ارزیابی هزینه‌بر است (به‌ویژه محاسبه‌ی تحرک که خود نیازمند تولید حرکت‌های قانونی است) و سرعت جست‌وجو را محدود می‌کند.

6. نحوه اجرا

برای اجرای بازی‌های نمونه میان عامل‌ها:

```
python main.py
```

برای اجرای کامل آزمایش‌ها (نرخ برد و مقایسه‌ی گره/زمان):

```
python run_experiments.py
```

یادآوری: فایل‌های محیط بازی و عامل‌های پایه تغییر نکرده‌اند. تنها `minimax_agent.py` و `alphabeta_agent.py` پیاده‌سازی شده و `main.py` برای نمایش عامل‌های جدید به‌روزرسانی شده است.